Querying with Conflicts of Interest

Nischal Aryal
Oregon State University
aryaln@oregonstate.edu

Arash Termehchy
Oregon State University
termehca@oregonstate.edu

Marianne Winslett
University of Illinois
winslett@illinois.edu

Abstract

Conflicts of interest often arise between data systems or service providers and their users regarding how the users' information needs should be interpreted by the system. For instance, an online product search service might be biased toward presenting certain products higher in its list of results to improve its revenue. As another example, data service providers may share users' queries with third parties or advertisement firms to model users' preferences, which users may view as breaches of their privacy. Data service providers have little to no incentive to implement strong measures for preserving users' privacy, such as encrypted databases or private information retrieval, or to reduce biases that increase their profits. Such methods often also increase query run times significantly. Hence, users may prefer to send queries to the data system that do not precisely reflect their true information needs, to achieve some degree of privacy. In this paper, we propose a novel formal framework that models the interaction between the user and data system in these settings. We investigate the settings in which it is possible for users to achieve their desired interpretations and results. We also propose efficient algorithms that enable users to get their desired interpretation and information, by reformulating their information needs.

CCS Concepts

• **Do Not Use This Code → Generate the Correct Terms for Your Paper**; *Generate the Correct Terms for Your Paper*; Generate the Correct Terms for Your Paper; Generate the Correct Terms for Your Paper.

Keywords

Do, Not, Use, This, Code, Put, the, Correct, Terms, for, Your, Paper

ACM Reference Format:
Nischal Aryal, Arash Termehchy, and Marianne Winslett. 2018. Querying with Conflicts of Interest. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX).* ACM, New York, NY, USA, 13 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Financial, social, and political factors prevent the incentives of data source providers and their users from being perfectly aligned. For example, data sources that provide information about products may benefit financially from convincing users to buy certain products rather than others. In fact, Amazon, Google, and Apple have been accused of returning results that are biased in this manner [14, 22, 24, 28], e.g., by only accepting changes to Amazon's search algorithm if the changes boosted Amazon's profits, and by placing advertised goods at the top of Google's search results while also giving a low ranking to legitimate comparison shopping sites.

This conflict of interest is inherent at sites such as Amazon, which charges sellers a 'referral fee' of 8-45% of the price of items sold on its site [1], so that it profits more when users buy higher-priced versions of a given product. Cost-conscious users who resort the results by price may have to wade through many irrelevant results before reaching the first good match, because Amazon search results often include many duplicative and/or minimally-related lower-cost items.

Example 1.1. For example, on the day of this writing, an Amazon search for 'headset' returns headsets priced $24.99 to $39.95 in its top 'Results' section, and all items on the remainder of the page are indeed headsets. When re-sorted by price, the top hit on the first page is a charging cord, followed by earbuds, microphones, cables, and one $1.98 headset. Only on the seventh page of results do headsets start to outnumber other accessories.

As another example, Apple allegedly reworked its ranking algorithm so that Apple Music was ranked first and Spotify 23rd in the App Store when a user searched for music apps [28]. Similarly, a search for podcast apps returned Apple Podcasts in first place, followed by 12 irrelevant Apple apps, and then finally podcast apps from other publishers. A search for "TV" in 2018-9 would return Apple apps at the top, and Netflix ranked in the 100s [28]. As experts commented at the time: "I find it hard to believe that organically there are certain Apple apps that rank better than higher-reviewed, more downloaded competitors," and "there's just a ton to be gained commercially by dominating" search results [28].

Further, many data source owners' business models involve leveraging users' queries for purposes other than answering them. Users may view this as breaching their privacy.

Example 1.2. Consider a user who searches a patent data source to find existing patents close to one she is now writing. If the data source owner uses, shares, or sells information about her queries, it can hurt the user's business [16, 19]. As another example, users often use location-based services to find shops or restaurants close to their current locations. The business model of these services is often based on sharing users' information with third parties. Users, however, do not necessarily want to share this information. Similarly, product data sources often share users' queries and preferences with other companies for financial gains, e.g., with advertisement firms to model users' interests and behaviors.

Conflicts of interest may also arise from non-financial considerations. To maximize user engagement, social media sites often

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, Woodstock, NY
© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

promote extreme content, which is not necessarily in anyone else's best interest. More generally, data sources may return information that is biased toward or against controversial topics and causes, political groups, or demographic groups, e.g., not returning or giving a low ranking to information relating to these [33, 34]. Perhaps the best known recent example is the removal of all topics related to climate change from US government websites [35].

In the aforementioned settings, data sources interpret the information in users' queries to take actions that do not exactly meet users' preferences and objectives and in fact might largely deviate from what users deem as desirable. Current proposals to combat undesirable data source actions usually require the data source to follow and implement certain protocols. For instance, current proposals for preserving users' privacy, such as encrypted databases [32], private information retrieval [10], and outsourced database model [26], require that the data sources implement methods that prevent them from recording or using queries for undesirable purposes. Similarly, researchers have proposed methods of bias elimination and fairness for data sources to implement [33, 34]. Also, these methods often increase the time of querying so significantly that users may not want to use them [10, 32].

Another option for dissatisfied users is to migrate to a data source owner whose incentives are better aligned with their own. For example, the patent searcher could choose to use the US Patent & Trademark Office's own free database, or a third party service that promises privacy for a fee. When querying the web, the user could adopt a privacy-friendly browser [13], or take advantage of other privacy-enhancing inventions. But what can the user do in our target setting, when a data source is a de facto monopoly for satisfying her information need?

In such a setting, the data source owners and users do share some interests: the owners would like to satisfy the information needs behind users' queries well enough to keep them engaged. Users can leverage this partial common interest by modifying their queries so that the owners' interpretations of the submitted queries are those desired by users.

Example 1.3. Consider again the user in Example 1.1 who looks for affordable headsets on Amazon. If the results are biased toward relatively more expensive products, she can add a price cap to her query. For example, when we reissued the Amazon headset query with a $19 maximum price, the *second* result returned cost $21, but the others on the first page cost below $19. Thus Amazon does not take price caps literally, but they do influence the search results.

Example 1.4. Users who search the patent data source in Example 1.2 might reformulate their queries to return supersets of their desired topics, to mask their true intent. Also, users in Example 1.2 who query location services may share noisy versions of their locations, e.g., nearby locations, with these services [3, 9, 15, 17, 25].

But two can play at this game. The data source may be aware of the fact that users know about its undesirable actions, e.g., users have observed undesirable biases in their previous experiences of using the data source. Thus, it may reason that users aim at guiding its interpretations and actions toward users' desired ones by modifying their original queries. Since it is not in the data source's best interest to accept the modified query as precisely the user's true information need, the data source may try to recover the original intent behind the modified query. For instance, it might also use its prior belief about users' intents, e.g., gained from external sources or previous interactions with users, to interpret the true intent behind the modified query. Consequently, users may reason about the data source reasoning process and try to modify their queries more drastically to influence the data source interpretation. The user and data source reason and make decisions based on possible reasoning and strategies of each other iteratively to find the best method to formulate and interpret queries.

Example 1.5. In Example 1.3, given that the user has added a degree of bias in the modified query, the data source may reason that the user has overstated their preference for headsets of a particular price, just to offset the data source's bias. Hence, it might avoid placing such headsets in the top positions, as we saw when Amazon still ranked a $21 headset second even after we limited the price to $19 in Example 1.1. At the extreme, the user might counteract this by further reducing her submitted maximum price for a headset.

Example 1.6. Consider again Example 1.2, where a user submits an overly broad query to a patent data source, to hide her intent. The data source may know that users might adopt such a strategy. Using prior information about the user, the data source might infer the user's intent from the submitted query. On the other hand, the user might consider that the data source knows about her desire to preserve privacy, reason about the data source's strategy, and submit an even broader query to further disguise her intent. Now, consider the user who aims to hide her true location to some degree from a location-based service in Example 1.2. The data source might be aware that the user might submit a noisy location and use its prior information to infer the true location of the user. To offset the data source reasoning, the user might add more noise to her submitted location. The user and the data source might continue this iterative reasoning about finding the right strategies to add noise to the query and remove it.

In some of the aforementioned settings, a brute-force approach for the user to get her desired information without allowing the data source to take undesirable actions is to submit a query that retrieves the entire or a large subset of the data and post-process that to extract her desired information. At the extreme, in Example 1.2, the user who searches a patent data source might submit a query that returns all patents. This is not usually practical due to the enormous amount of data managed by these data sources. Furthermore, data sources might not return such a large dataset as it is against their objective, e.g., they might ask users to resubmit a query that returns a relatively small subset of the data. Also, data sources usually limit the rate of retrieved information, which makes it difficult for users to download a very large dataset using multiple queries. Moreover, users often lack the computational resources, time, and skills to store and post-process a large dataset.

There are three important questions in this monopolistic setting. First, we would like to know under what conditions the aforementioned recursive reasoning of the user and data source will converge to a stable state in which the user finds a query that gives her useful information, i.e., *informative communication*. The user might be able to leverage the fact that the data sources would like to keep the user engaged, i.e., partial common interest, to convince

the data source to perform her desired actions to some degree. For instance, in cases where the user queries a biased data source, we would like to know when the user eventually finds a query to express their true intent such that the system is convinced to return some less biased answers. Similarly, in the setting where users would like to preserve their privacy, e.g., Example 1.2, we would like to know in which cases the recursive reasoning of the user and system about each other's strategies will converge to a stable state in which the user composes a query that provides her with a given level of privacy.

Second, we would like to know how close users can get to their ideal answers. For instance, if a user knows that the system is biased, how much of her desired information can she extract from the system by reformulating her queries to account for that bias? Third, we would like to find efficient methods to determine the best query for the users to submit.

In this paper, we propose a framework that models strategic information sharing between users and data sources when there are conflicts of interest between users and data sources. It models the user and data source as agents with different objectives. Our framework leverages the current body of research in game theory on how agents with different goals establish informative communication [11].

Our contributions are as follows.

- We show that the recursive reasoning of the user and system will always converge to stable states given mild conditions on their objective functions.
- We show that the amount of desired information that the user receives in these stable states depends on the amount of conflict of interest between the user and the system. We also show that users can use certain constructs in query languages to pick the stable states in which they receive more desired information.
- We design efficient algorithms for the user to find their maximum possible desired information.

2 Framework

2.1 Data Model & Query Language

A schema $\mathcal{R}$ is a set of relation symbols $R_i, 1 \leq i \leq n$, where each R_i has a fixed arity z_i. Let **dom** denote the universe of possible data values, and for each attribute position j of R_i let $\mathbf{dom}(A_{i,j}) \subseteq \mathbf{dom}$ be its specific domain. Each tuple of the relation symbol R_i is then an element of the Cartesian product $\prod_{j=1}^{z_i} \mathbf{dom}(A_{i,j})$. A relation of R_i is a finite set of such tuples. An instance of $\mathcal{R}$ is a set of relations for its relation symbols.

Let $\mathcal{Q}$ denote a query language. An z-ary query $q \in \mathcal{Q}, z > 0$, in schema $\mathcal{R}$ is a function that maps each instance I of $\mathcal{R}$ to a partially ordered list where each member of the list belongs to $\mathbf{dom}^{\mathbf{z}}$. We denote the result of query q over I as $q(I)$. The order among the tuples in $q(I)$ is based on the user's preferences, which indicates the degree by which the user is interested in a tuple.

More formally, given an z-ary query q over instance I, the **user preference** is a function $\Theta_{(q,I)}$ that maps tuples in $\mathbf{dom}^{\mathbf{z}}$ to $[0, 1]$. We drop (q, I) from $\Theta_{(q,I)}$ when the query and instance are clear from the context. TODO: Remove extra elm notation Let $elm(l)$ denote the set of tuples in a partially ordered list l. We use $e \prec e'$ ($e \sim e'$) to show that e precedes e' (e and e' share the same position) in l. Given the user preference $\Theta_{(q,I)}$, if $e \in elm(q(I))$ ($e \notin elm(q(I))$), we have $\Theta(e) > 0$ ($\Theta(e) = 0$). Also, if $e.e' \in elm(q(I))$ and $e \prec e'$ ($e \sim e'$), we have $\Theta(e) > \Theta(e')$ ($\Theta(e) = \Theta(e')$). Queries that return an unordered set of tuples are a special case of these definitions.

Users can express queries in our framework using query languages, such as SQL, that return a set or a ranked list of results, e.g., using ranking constructs such as Order By Case clause [27, 29, 31]. They can also use proposed constructs to express partial preference relationships in query languages [18, 23, 30].

Example 2.1. Consider an electronics shopping site (Table 1), where each headphone tuple has attributes `brand`, `quality` and `price`, where `quality` and `price` are numerical. User may submit queries such as `SELECT DISTINCT brand, quality FROM Headphones ORDER BY quality DESC`. They may also submit a query with a different ranking of headphones: `ORDER BY CASE quality WHEN 2 THEN 0 WHEN 3 THEN 2 ELSE quality END DESC`

Table 1: Headphones

id	brand	quality	price
e_1	A	3	89.99
e_2	B	2	69.99
e_3	O	1	59.99
e_4	C	0	39.99

2.2 Actions

User actions. The user has a true information need, i.e., their *intent*, which is expressed in the form of a query $\tau \in \mathcal{Q}$. The data source does *not* know the user's intent. The user picks their intents from a prior distribution $\Pi(\tau)$, which is known to the data source. For example, when a user asks Amazon for headphones, Amazon knows which headphones customers like her have bought in the past, and can estimate that likelihood for new products. Given intent τ, the user sends queries from $\mathcal{Q}$ to the data source. The submitted query q might be different from τ, e.g., to hide the user's intent τ or offset the bias in the returned results of q.

Data source actions. The data source *interprets* the query submitted by the user, $q \in \mathcal{Q}$, to get its own desired query β. If there is a conflict of interest between the user and the data source, the interpreted query β might not be the same as q. For instance, in the settings where the user submits a different query from their intent to achieve some degree of *privacy*, the data source aims at finding a β that is equivalent to τ, i.e., produces the same results as τ. Similarly, if the data source is biased towards returning certain information, β is generally not equivalent to q.

2.3 Strategies

User strategy. The *user strategy* $P^r(q|\tau)$ is a (stochastic) mapping from $\mathcal{Q}$ to $\mathcal{Q}$. Given the intent τ that represents the user's true information need, P^r returns a query that the user submits to the data source.

Example 2.2. Consider the scenario in Example 2.1. The user would like to observe a wide range of headphones with different

degrees of quality to make an informed decision. Thus, they may submit a query $q_{quality}$ that expresses their *intent* (τ) to return a list of all headphones on the shopping site, ordered by the values of the *ranking attribute* quality (`ORDER BY quality DESC`). On the other hand, user may also decide to hide their intent and submit a query $q'_{quality}$ `ORDER BY quality ASC`. Similarly, in our setting, the user strategy is a mapping from their intent to queries.

Data source strategy. The *data source strategy* $P^s(\beta|q)$ is a (stochastic) mapping from Q to Q. The data source uses P^s to interpret the intent behind the submitted query q and acts accordingly. For instance, if the data source is biased, given q, it computes the biased query β and returns its results on the database instance. In the settings where the user submits a variation of their query to preserve privacy and hide their true intent, the data source aims at finding a β that is equivalent to τ to discover the user's intent. As the goal of the system in this case is *not* to return biased results, it will return the results of the submitted query q to the user. In this paper, we focus on the common aspect of these settings, which is how the data source finds its desired β from the submitted query q by computing appropriate strategies.

Example 2.3. Continuing with Example 2.2, assume that the higher quality a headphone is, the more expensive it is. The shopping site makes more profit from selling more expensive headphones; therefore, in response to $q_{quality}$ it returns a ranked list of headphones whose qualities are above a relatively high quality threshold ϵ_l (`quality > 3`). Generally, given a query q, if the position of a tuple in the ranked list or its presence in the result is according to a condition based on *quality* (*price*), the source returns the tuple if its *quality* (*price*) is above ϵ_l (ϵ_p which represents a relatively high price), respectively. This condition biases the results towards **not returning** headphones e_3 and e_4 in Table 1. As another form of bias, the data source may prefer products owned by itself (*brand*) due to higher profitability. Therefore, weighing each tuple by its brand w_b, assigning higher weights to its brand (`brand=0`). With this condition data source would return e_3 before any other tuple, therefore **misranking** certain tuples.

2.4 Utility functions

The user's and data source's **utilities measure the desirability** of the data source's interpretation of the user's intent for the user and data source, respectively. Let $U^r(\tau, \beta)$ and $U^s(\tau, \beta)$ denote the user and data source utility functions, respectively, which map pairs in $Q \times Q$ to real values.

If there is some conflict of interest between the user and system, U^r and U^s take their maximum values for different values of β for the same intent τ. For example, if a user would like to preserve some level of privacy by submitting a query that is different from their intent, $U^r(\tau, \beta)$ takes its maximum for a $\beta \neq \tau$ while $U^s(\tau, \beta)$ takes its maximum for a β that is equivalent to τ. On the other hand, if the user queries a *biased data source*, U^r takes its maximum when β and τ are equivalent, and U^s will be at its maximum for $\beta \neq \tau$.

Example 2.4. Consider the intent query τ from Example 2.2 but *no access to the underlying tuples in the Headphone DB instance.* Assume only two possible system interpretations: β_1 : `ORDER BY quality ASC` and β_2 : `ORDER BY CASE WHEN quality=3 THEN 1 WHEN quality=1 THEN 3 ELSE quality END`. Since β_1 fully reverses τ while β_2 preserves some preferences, β_2 better matches the user's preferences. To output a real-valued score, suppose the utility function measures absolute errors in ranking positions. Concretely, assume access to the domain of `quality` based on a known schema $\mathcal{R}$, the true ranks under τ are $[1, 2, 3, 4]$ and under β_1 are $[4, 3, 2, 1]$, so, $U^r(\tau, \beta_1) = -\big(|1-4|+|2-3|+|3-2|+|4-1|\big) = -8$, Similarly, $U^r(\tau, \beta_2) = -2$,, therefore interpretation β_2 provides higher utility to the user.

Data sources maintain a large amount of data, and most users do not know the full content of the databases. Therefore, as we illustrate in Example 2.4, we focus on utility functions that measure the *conflict between queries themselves* rather than the result list of queries. To measure utilities over all possible instances of the data, we rely on schema-level information (e.g. attribute domains). By contrast, with unrestricted access to all the tuples, one may define instance-dependent utilities. For example, one could define utility as a function (e.g sum) of actual `quality` values of the tuples returned by each query on a specific database instance

2.5 Belief and Equilibrium

As we explain in Section 1, the user and the data source use their available information to reason about the strategies of the other party to maximize their utility function. We would like to identify whether this reasoning will converge to any eventual stable state in which the user and data source settle on a pair of fixed strategies. Knowing these states will enable us to analyze the strategies selected for the user and the data source and design algorithms to find desired strategies for users in these settings efficiently.

Example 2.5. Continuing with Example 2.3, assume that the user prefers to hide their intent and submits $q'_{quality}$ to the data source. The data source again misranks some tuples in the result to increase its profit. The user may decide to submit a different query $q''_{quality}$ to overcome the bias of the data source or stop using the data source altogether. The user could also be willing to accept such misrankings as long as the tuple highly ranked in their intent (e_1) is also present or ranked high enough in the result list. Similarly, the data source also wants to keep users engaged and, therefore, is often willing to partially satisfy the user's request. Therefore, although the data source misranks e_3 in the result list, it may still return e_1 after e_3 but drop e_2 or add e_4 at the end of the list. In either case, the user provides partial information about their intent, and the data source partially satisfies the user's information need. Therefore, the user and data source may settle on a pair of fixed strategies.

Since the data source does not have complete information about the user's intent, we follow the approach used in the game theory literature and use the concept of *Bayesian equilibrium* to characterize the stable state of the user and system reasoning [11]. **The belief of the data source** $\Phi(\tau|q)$ is a stochastic mapping from Q to Q. The belief of the data source maps each user's submitted query q to a probability distribution over the set of possible intents. **The expected utility of the user** for the query q is

$$\mathbb{E}[U^r(\tau, \beta) \mid q] = \sum_{\beta \in Q} U^r(\tau, \beta) P^s(\beta \mid q).$$

The expected utility of the data source for the query q is

$$\mathbb{E}[U^s(\tau, \beta') \mid q] = \sum_{\tau \in Q} U^s(\tau, \beta')\Phi(\tau \mid q).$$

where $\Phi(\tau \mid q) = \frac{P^r(q|\tau)\pi(\tau)}{\sum_{\gamma \in Q} P^r(q|\gamma)\pi(\gamma)}$. The data source updates its belief on the user's intent based on P^r and prior π using the Bayes rule.

Definition 2.6. An equilibrium is a pair of user and data source strategies $(P^r(q|\tau), P^s(\beta|q))$ such that

- for each τ, if $P^r(q \mid \tau) > 0$, $q = \arg\max_{q'} \mathbb{E}[U^r(\tau, \beta) \mid q']$.
- for each q, if $P^s(\beta \mid q) > 0$, $\beta = \arg\max_{\beta'} \mathbb{E}[U^s(\tau, \beta') \mid q]$.

In an equilibrium, the pair of user and data source strategies provides them with the highest utility (**optimal**) given the strategy of the other party. A well-known equilibrium for this interaction in the literature of game theory is the babbling equilibrium [11], i.e, a uninformative one.

Definition 2.7. Let $\beta = \arg\max_{\beta'} \mathbb{E}[U^s(\tau, \beta')]$ be the interpretation that is optimal for the data source based only on its prior belief π. An equilibrium $(P^r(\cdot|\cdot), P^s(\cdot|\cdot))$ is a **uninformative equilibrium** if for every query q that the user might send, the data source's strategy is to always return this same interpretation. Formally, for every q sent with positive probability, $P^s(\beta|q) = 1$. Otherwise the equilibrium is **informative**

Intuitively, in an uninformative equilibria, the data source finds it optimal to compute the preference values of candidate tuples and return the answers purely according to its prior belief. At the most basic level, this happens if the user does not send any query or sends a meaningless query to the data source. In Section 3, we discuss more interesting conditions under which uninformative equilibria arise in our setting.

3 Credible Interaction

Users may strategically send queries that misrepresent their intent to convince the data source to return their desired information. However, the effectiveness of users' strategies may depend on their interpretation by the data source and the data source's degree of bias. In the following sections, we first define the space of all queries that users may submit (Section 3.1). Then, we prove how extreme misalignment between the user's and system's utility functions leads the communication to an uninformative equilibrium (Section 3.2). Finally, we present an algorithm to detect queries that are *not effective* in influencing the data source's interpretations (Section 3.4).

3.1 Possible Reformulations

We assume users' intent query $\tau \in Q$, induces a strict total order on the returned tuples (Section 2.1). When there is conflict between users' and data source's utility function (Section 2.4), users may submit modifications (reformulations) of their intent. Formally, we define a reformulated query as follows:

Why already assume already no dropped tuples or new tuples? Comparative cheap talk works with fixed N we can come back to this.

Definition 3.1. Let $q, q' \in Q$ be two queries of identical arity. We call q' a **reformulated query** of q if it changes the ranking order of tuples with respect to q for *at least one database instance* I. Equivalently, there must exist at least one pair of tuples (e_i, e_j) satisfying: $e_i \prec e_j$ in $q(I)$, but $e_j \prec e_i$ or $e_j \sim e_i$ in $q'(I)$. We denote the set of all such reformulated queries by $Q_{\text{RF}}(q)$.

Example 3.2. Some possible reformulations of the intent query τ in Example 2.2 are {`ORDER BY quality ASC`, `ORDER BY quality CASE WHEN 3 THEN 1, WHEN 1 THEN 2, ELSE 0` ...} $\in Q_{\text{RF}}(\tau)$. Specifically, for $\tau(I)$ where I is the instance in Table 1 the set $Q_{\text{RF}}(\tau)$ contains reformulated queries resulting in: (1) *Partial orders*: $e_1 \prec e_2 \sim e_3 \prec e_4$ or $e_1 \sim e_3 \prec e_2 \sim e_4$ or $e_1 \sim e_2 \sim e_3 \sim e_4$; (2) *Total orders*: $e_1 \prec e_2 \prec e_4 \prec e_3$ (adjacent swap), $e_2 \prec e_1 \prec e_3 \prec e_4$ (non-adjacent swap), or $e_4 \prec e_3 \prec e_2 \prec e_1$ (reverse order). Each alters at least one preference order from the original, e.g., $e_3 \sim e_4$ replaces $e_3 \prec e_4$, or $e_4 \prec e_3$ replaces $e_3 \prec e_4$.

3.2 Impossibility of Informative Equilibria

A user's strategy (Section 2.3) may pick any reformulated query from $Q_{\text{RF}}(\tau)$ to submit. In theory, a reformulated query can boost user utility; however, a rational data source has no incentive to accept *every* query the user submits as precisely the user's true intent (Section 1). As discussed in Section 2, in this work, we assume the system is also a reasoning agent. Consequently, the data source may be aware that users are aware of its undesirable actions, e.g., users have observed undesirable biases in their previous experiences with the data source. Therefore, modern data sources may actively detect and counter suspicious query reformulations. As Example 3.3 illustrates, this can lead to adversarial interpretations that undermine users' strategies. In turn, leading to an endless cycle of reformulations and reasoning, preventing the user and data source from settling on any fixed, stable strategies (Section 2.5).

Example 3.3. Consider a scenario where the user's intent is to search for top-quality headphones from the instance in Table 1. Suppose the user's strategy (Section 2.3) maps the intent to the query q: `WHERE brand IN {A, B} ORDER BY quality DESC` $\Rightarrow e_1 \prec e_2$. Assume the system has the threshold utility from Example 2.2 , $U^s(\tau, \beta) = \sum_{d \in \{1,2\}} (\Theta_\tau(e_d) - \epsilon_d)\beta(e_d)$ with the following bias: $\epsilon_{e_1} = 0.75, \epsilon_{e_2} = 0.55$ and the user has the same utility function without bias, $U^r(\tau, \beta) = \sum_{d \in \{1,2\}} \Theta_\tau(e_d)\beta(e_d)$. Suppose the user's preferences (Θ) are uniformly distributed over $[0, 1]$ and therefore computes expected posteriors as $p_i = \mathbb{E}[\Theta(e_i) \mid q] = \frac{k-i+1}{k+1}$, where k is the size of the domain of quality. That is, $p_1 = 0.66, p_2 = 0.33$. Thus, no tuple passes the threshold condition and none would be returned. To retrieve their preferred tuples, the user has an incentive to deviate from their strategy and submit a reformulated query (Def. 3.1) q': `WHERE brand IN {A,B} ORDER BY quality ASC` $\Rightarrow e_2 \prec e_1$ In this scenario, the user is expecting the posterior estimate to reverse: $p_1 = 0.33$, $p_2 = 0.66$, resulting in e_2 meeting the threshold condition, and being returned. However, the data source can reason about the user's possible intents and manipulations. The data source knows that the user is aware of its bias and may be trying to game the system. Therefore, q' could equally represent the user's true intent or a deliberate manipulation (Example 3.10). Given this uncertainty, it is in the data source's best interest to ignore the query and revert to its prior belief about the

user's intent. Based on the uniform prior, $p_1 = 0.5, p_2 = 0.5$, and thus no tuples are returned. The user also anticipates this reasoning from the data source, prompting a new cycle of reformulations and interpretations.

In general, user and system strategies may fail to converge to an informative equilibrium due to two main reasons: (i) agents' reasoning about each other's strategies and (ii) the degree of conflicts between the user's and the system's objectives. Next, we place some structural assumptions on the utility functions and the preference function.

Additivity of Utility Functions: We assume that for any intent τ and interpretation β, the data source's utility is **additive**, meaning they decompose as a sum of per-tuple utilities:

$$U^s(\tau, \beta) = \sum_e u_e^s(\Theta_\tau(e), \Theta_\beta(e)) \tag{1}$$

Where u_e^s is a function from pairs of preference values (Section 2.1) to real values This assumption simply expresses the system utility function as a sum of utility functions, each based on the relative ranking positions in τ and β. We make similar assumptions about the structure of the user's utility function. This structure is satisfied by the utility functions in both Lemma 3.5 and Lemma 3.6.

Independence of Preference Function: We assume the user's preferences over tuples are i.i.d. under the prior (e.g., Uniform$[0, 1]$); that is, the preference function Θ draws each value independently from the same distribution (Section 2.1).

For the rest of the paper, we assume the utility functions are additive (Eq. 1) and the preference function is i.i.d. We first show that the i.i.d. assumption leads to a useful property on the data source's posterior beliefs (Section 2.5) about users' preferences.

Lemma 3.4. *If Θ maps tuples independently to an identical prior distribution, then for any query q, the expected posterior for each tuple depends only on its position in the ranking.*

Lemma 3.4 says that upon receiving a query, the data source's updated belief (Section 2.5) about the user's preference for a tuple is independent of the specific ordering of the other tuples Now we use this property to prove that extreme misalignment between the user's and system's utility functions leads to uninformative equilibria. We first consider a linear utility function with a threshold condition that generalizes the interaction in Example 3.3.

Lemma 3.5. *Consider an interaction over two tuples, e_i and e_j. Let the user's utility be, $U^r(\tau, \beta) = \sum_{d \in \{i,j\}} \Theta_\tau(e_d)\beta(e_d)$. Let the System's utility be $U^s(\tau, \beta) = \sum_{d \in \{i,j\}} (\Theta_\tau(e_d) - \varepsilon_d)\beta(e_d)$. Let $E_H = \mathbb{E}[\Theta(e_d)|e_d$ is ranked high$]$ and $E_L = \mathbb{E}[\Theta(e_d)|e_d$ is ranked low$]$ be the posterior expected preferences. If the System's preference thresholds are not equal such that the threshold for either tuple falls outside the range $\varepsilon_d \notin (E_L, E_H)$ for some $k \in \{i, j\}$, then the only possible equilibrium is uninformative.*

Lemma 3.6. *Consider an interaction over two tuples, e_i and e_j. Let the user's utility be a quadratic loss function, $U^r(\tau, \beta) = -\sum_{d \in \{i,j\}} (\beta(e_d) - \Theta_\tau(e_d))^2$. Let the System's utility also be a quadratic loss function but with a bias b_d: $U^s(\tau, \beta) = -\sum_{d \in \{i,j\}} (\beta(e_d) - (\Theta_\tau(e_d) + b_d))^2$. If the System's biases are not constant ($b_i \neq b_j$), then the only possible equilibrium is uninformative.*

Using Lemma 3.4, we can extend the results from Lemmas 3.5 and 3.6 to real world scenarios where data sources maintain many tuples.

Theorem 3.7. *Consider an interaction over finitely many tuples and the additive utilities of Lemmas 3.6 and 3.5. Suppose that for every pair of tuples (e_i, e_j), the conditions of either Lemma 3.6 or 3.5 hold. Then the only possible equilibrium is uninformative.*

Intuitively, Theorem 3.7 uses Lemma 3.4 to break down the many tuple problem into pairwise sub-problem. Since each pairwise interaction can only lead to an uninformative equilibrium, any interaction result in an uninformative equilibrium (Definition 2.7).

3.3 Credibility of Querying

In Section 3.2, we demonstrate conditions on certain utility functions for the interaction to collapse to an uninformative equilibria. In this section, we identify general **properties of submitted queries** that lead to uninformative equilibria.

Credible Queries: We first define the notion of credible queries. Consider two tuples e_i and e_j. The user's true intent is one of two strict preferences: either $e_i \prec e_j$ or $e_j \prec e_i$. Let τ and τ' denote two intents such that $\Theta_\tau(e_i) > \Theta_\tau(e_j)$ and $\Theta_{\tau'}(e_j) > \Theta_{\tau'}(e_i)$. Let P^r be the user's strategy (Section 2.3) that maps intents to submitted queries.

Definition 3.8. A pairwise order $(e_i \prec e_j)$ is **credible** if user intents with opposing preferences do not send the same query: there is no single query q such that $P^r(q \mid \tau) > 0$ and $P^r(q \mid \tau') > 0$ while q asserts $(e_i \prec e_j)$. Otherwise, the order is **non-credible**.

Definition 3.9. For any query $q \in Q$, let $\mathrm{OP}(q)$ be the set of all *strictly* ordered pairs asserted by q

$$\mathrm{OP}(q) = \{(e_i \prec e_j) | e_i, e_j \in q \text{ and } e_i \text{ is ranked higher than } e_j\}$$

A query q is **credible** if it contains at least one credible order i.e., there exists $(e_i \prec e_j) \in \mathrm{OP}(q)$ that is credible. Otherwise, q is **non-credible**.

Intuitively, a credible query is one where the system can trust that the user has provided at least partial information about their true intent.

Example 3.10. Consider the same scenario as Example 3.3. The data source receives the user's submitted query $q' : e_2 \prec e_1$. The data source does not know the user's true intent; however, it knows that the user has two possible intents τ_1: $e_1 \prec e_2$ and τ_2: $e_2 \prec e_1$. The data source derives that the user strategy P^r maps both intents to the same query $q' : e_2 \prec e_1$ to get higher utility. Hence, the order $(e_2 \prec e_1)$ is *not credible*. Since $\mathrm{OP}(q') = \{(e_2 \prec e_1)\}$ contains no credible order, the query q' is *non-credible*.

Proposition 3.11. *If the user's strategy is to submit a non-credible query (Definition 3.9) for every intent, then the only possible equilibrium is uninformative.*

Proposition 3.11 formalizes the **reasoning of the data source** after the user submits a query q. The data source does not have complete information on the user's true intent τ. To interpret the query, the system must determine if the preference orders asserted in q are credible. If the system reasons that the user sends the same

query regardless of their true intent, the query is equivalent to a meaningless query for the system. The system then behaves exactly as under the prior, resulting in an uninformative equilibrium (Definition 2.7).

3.4 Detecting Credibility of Querying

Based on the results of Proposition 3.11, we now develop an efficient test for query credibility. To build intuition, we focus on the case where the submitted query q ranks tuples by a single attribute A in schema $\mathcal{R}$ (e.g., `ORDER BY quality`), and the system's bias is also a function of A. We then discuss extensions to SPJ queries and more complex bias models; the core ideas carry over but require additional inputs beyond $\mathcal{R}$ (TODO: elaborate).

Problem: Given a submitted query q, schema $\mathcal{R}$, and utility functions (U^r, U^s) as **input**, the problem is to **output** whether the query is credible (Definition 3.9), and if so, the set of credible orders (Definition 3.8). We output the set of credible orders to use them in subsequent algorithms in Sections 4 and 5.

Definition 3.12. Let $q, q' \in \mathcal{Q}$. We call query q' a **swap query** of q if it switches the position of exactly one strictly ranked pair of tuples in q and all other tuple positions remain exactly as in q. We denote the set of all possible swap queries as $\mathcal{Q}_{\text{swap}}(q)$.

Intuitively, after the user's strategy $P^r(q|\tau)$ has picked a refomulation of their true intent τ and submitted it as query q (possibly $q \neq \tau$), $q' \in \mathcal{Q}_{\text{swap}}(q)$ is a refomulation of q. For instance, in Example 3.10, a possible swap query of $q' : e_2 \prec e_1$ is $q'' : e_1 \prec e_2$, which is a possible user intent τ_1.

Algorithm: Algorithm 1 determines the credibility of the input query q by iterating through every possible *swap query* $q' \in \mathcal{Q}_{\text{swap}}(q)$ (Definition 3.12). Each q' isolates a single pairwise order for testing credibility (Proposition 3.13). For each test, the algorithm first determines the system's interpretation for the original query q and the swap query q'. This is done using the arg max function:

$$\beta_q \leftarrow \arg\max_{\beta'} \ \mathbb{E}[U^s(\tau, \beta') \mid q]$$

However, for utility functions with multiple maxima, the arg max may not always be unique. That is, the system could be indifferent between several optimal interpretations (β) that provide the same maximal utility. To resolve this ambiguity, we assume a consistent *tie-breaking rule*: if multiple interpretations $\{\beta_1, \beta_2, \dots\}$ yield the same maximal utility, the system chooses the one most favorable to the user. Since the source does not know the user's true intent τ, it selects the interpretation that is closest to the user's query, i.e, β_d that maximizes $U^r(q, \beta_d)$. This is a reasonable assumption since the data source wants to keep the user engaged.

With the system's responses (β_q and $\beta_{q'}$) determined, the algorithm simulates users' possible strategies. It considers two possible user intents: one that matches the original query, $\tau^{(q)}$, and one that matches the swap query, $\tau^{(q')}$. Then, it evaluates the user's utility for both intents under both interpretations. Intuitively, the procedure populates a 2×2 matrix with the utility each user intent would receive from submitting each query:

	submit q	submit q'
$\tau^{(q)}$	$U^r(\tau^{(q)}, \beta_q)$	$U^r(\tau^{(q)}, \beta_{q'})$
$\tau^{(q')}$	$U^r(\tau^{(q')}, \beta_q)$	$U^r(\tau^{(q')}, \beta_{q'})$

The order in input query q (e,g $e_i \prec e_j$) is added to the set of **credible orders** C unless both user intents ($\tau^{(q)}$ and $\tau^{(q')}$) *strictly* prefer to submit the exact same query (*both* rows have high value on the same column).

Algorithm 1 Credibility Detection

Input: submitted query q, schema $\mathcal{R}$, utilities U^r, U^s
Output: credibility status, set C of credible orders
$C \leftarrow \emptyset$
$\beta_q \leftarrow \arg\max_{\beta'} \ \mathbb{E}[U^s(\tau, \beta') \mid q]$
for each $q' \in \mathcal{Q}_{\text{swap}}(q)$ **do**
  $\beta_{q'} \leftarrow \arg\max_{\beta'} \ \mathbb{E}[U^s(\tau, \beta') \mid q']$
  Let $\tau^{(q)}$ and $\tau^{(q')}$ be the two intents that differ only on the strict rank relation reversed between q and q'.
  $u_q^{(q)} \leftarrow U^r(\tau^{(q)}, \beta_q)$; $u_{q'}^{(q)} \leftarrow U^r(\tau^{(q)}, \beta_{q'})$
  $u_q^{(q')} \leftarrow U^r(\tau^{(q')}, \beta_q)$; $u_{q'}^{(q')} \leftarrow U^r(\tau^{(q')}, \beta_{q'})$
  both_prefer_q $\leftarrow \left(u_q^{(q)} > u_{q'}^{(q)}\right) \wedge \left(u_q^{(q')} > u_{q'}^{(q')}\right)$
  both_prefer_qprime $\leftarrow \left(u_{q'}^{(q)} > u_q^{(q)}\right) \wedge \left(u_{q'}^{(q')} > u_q^{(q')}\right)$
  if $\neg$(both_prefer_q $\vee$ both_prefer_qprime) **then**
    $C \leftarrow C \cup \{$ original order in input q $\}$
return (Credible, C) if $C \neq \varnothing$; else (Non-Credible, $\emptyset$)

Complexity. The credibility detection algorithm (Algorithm 1) iterates over all swap queries (Definition 3.12). In the worst case (when q is a total order) resulting in a search space of size $|\mathcal{Q}_{\text{swap}}(q)| = O(k^2)$, where $k = |\mathcal{D}|$ is the size of the domain of the ranking attribute A in query q. Within each iteration, the algorithm performs a fixed number of operations, primarily calls to compute the utility values and system responses. Assuming these underlying computations can be performed efficiently, the overall runtime complexity of Algorithm 1 is $O(k^2)$.

Proposition 3.13. *Using the results from Lemma 3.4, Algorithm 1 correctly identifies whether q is credible (Definition 3.9) and returns exactly the set of credible orders (Definition 3.8).*

4 Informative Equilibria

We have established that a user's query can only be part of an informative equilibrium if it is credible (Definition 3.9). In this section, we first study conditions that guarantee that the user and the data source's reasoning converge to an equilibrium (Definition 2.6). Then we characterize the space of all queries that can be part of an informative equilibrium (Definition 2.7).

4.1 Partial Common Interest

We first place some mild and natural restrictions based on the preference functions of queries to the utility functions of the user and system. Our condition on the utility functions ensures that the user

has some incentives to submit their information needs to the system to get relevant information.

Definition 4.1. Let U^s denote the system's utility function from Equation 1. U^s is (strictly) **supermodular** if for any two interpretations β, β' with $\Theta_\beta(e_d) < \Theta_{\beta'}(e_d)$, the function $u_e^s\big(\Theta_\tau(e_d),\ \Theta_\beta(e_d)\big) - u_e^s\big(\Theta_\tau(e_d),\ \Theta_{\beta'}(e_d)\big)$ is (increasing) non-decreasing in $\Theta_\tau(e_d)$. Equivalently, for a single tuple, assigning it to a higher-ranked position cannot lead the system's interpretation to assign it a lower position.

Partial Alignment: Intuitively, the supermodularity of utility functions ensures that the system places tuples that are highly ranked in the user's intents in high positions in its interpretation. Hence, it establishes some level of agreement across the rankings between the user intent and its interpretation by the data source, so the user has some incentives to use the system.This structure of the utility function is well-studied in game-theory [2, 8],economics [11, 21] and information retrieval [6, 7]. We define supermodularity for the user utility function similar to the one in Definition 4.1. Supermodularity of the user utility function expresses that the user prefers results that match their preferences the most. For the rest of the paper, we assume that the utility functions of the user and data source are supermodular.

Example 4.2. Consider a Headphone DB in Table 1. Suppose, due to a bias toward promoting expensive headphones for higher revenue, the data source's utility function assigns higher values to results that rank pricier headphones at the top, or that return expensive models in set-based queries. If this utility function is *not* supermodular, then when a user submits a query for a specific brand, the system may disregard the query and return only the most expensive headphones, whether or not they satisfy the input query. Users would avoid such a data source, as its returned answers often completely disregard their input query. In contrast, a supermodular utility function ensures that the system returns precisely the set (or a subset) of headphones satisfying the selection condition and that the results are mostly sorted in decreasing order of quality.

Partial Conflict: However, supermodularity is a property of individual tuples and does not imply that the user and data source have perfectly aligned interests. This is because the function $u_i^s(.,.)$ ($u_j^s(.,.)$) in Equation 1 vary across tuples. For example, in Example 2.3, the bias term in the system's utility varies across different headphone brands. Due to biases (thresholds or filters), the system can (1) **remove** certain tuples. (2) may **reorder** certain tuples in the result list. We observe these features in Examples 3.3, Lemmas 3.5 and 3.6, where utility functions are supermodular, but due to the system's bias, it's interpretations conflict with the user's preferences. However, often the degree of conflict is small to allow the user to keep using the system. If the system's bias is small, the user might be indifferent to the bias. Supermodular utility functions can also model this partial conflict.

Example 4.3. Suppose the user's true intent is the same as Example 2.2 ($e_1 \prec e_2 \prec e_3 \prec e_4$) and the user submits a reformulated query $q(I) : e_2 \prec e_1 \prec e_3 \prec e_4$. Assume that the user has a supermodular utility function $U^r = \sum_k -\max(|\Theta_\tau(e_k) - \Theta_\beta(e_k)| - 1, 0)$. The system has a supermodular quadratic utility function (Lemma 3.6) with a bias that always promotes tuple e_3 higher. Due to its bias, the system produces an interpretation that does not match the user's intent: $\beta(I) : e_1 \prec e_3 \prec e_2 \prec e_4$. For e_3, the true rank is 3, but the system placed it at 2. The error is $|2-3|$, and the utility is still at the maximum value of 0. The user is perfectly content with the reordered outcome and will keep using the system.

4.2 Possible Informative Equilibria

In Section 3.4, we investigated how non-credible queries (Definition 3.9) can lead to an uninformative equilibrium (Definition 2.7) and provided an algorithm (Algorithm 1) to identify the credible information in a user's query. In this section, we study the types of equilibria possible because of the credible information in a user's query.

Example 4.4. Consider a user's submitted query $q : e_1 \prec e_2 \prec e_3 \prec e_4$. Assume Algorithm 1 outputs $C = \{(e_1 \prec e_2), (e_2 \prec e_3), (e_3 \prec e_4)\}$. An inconsistency exists because the credible pairs form a path $e_1 \prec e_2 \prec e_3$, which by transitivity implies the preference $(e_1 \prec e_3)$. However, this implied preference is not in C and is therefore non-credible.

Inconsistencies in the query: Section 3.4 describes a pair-wise test that outputs the set of all credible pairwise preferences C from a user's query q. However, this set of pairs does not yet constitute a single query and may contain logical inconsistencies (Example 4.4).

One approach to supporting an equilibrium in the presence of such inconsistencies, well-studied in game-theory literature, is to allow the user to submit multiple disjoint preferences [11, 21]. In our setting, this is equivalent to the user submitting **multiple queries** (e.g., $q_1 : e_1 \prec e_2$ and $q_2 : e_3 \prec e_4$), one for each credible order in C, and the system correctly interpreting this and returning multiple disjoint results. ADD: multiple queries as part of a single request-ambiguous case, large data sources often do not support these kinds of queries Another approach is to **increase the expressiveness** of the query language (e.g., natural language) to allow users to express such complex disjoint preferences in a single query. However, our current data model (Section 2.1) is focused on single queries expressed using SQL constructs, and extending it to multiple queries and other expressive languages is an interesting direction for future work. Therefore, to formulate the set of all possible equilibria, we first construct a single query from the set of credible orders C. Ideally, we would like to find a reformulated query (Definition 3.1) that preserves the relative preferences expressed in the original query q, i.e., is in some sense close to q.

Definition 4.5. Let q be any submitted query. Let $C(q)$ be the set of credible orders (Definition 3.8) returned by Algorithm 1 when run on q. A **base credible query** q^{base} is a query obtained from q by merging adjacent tuples (or sets of tuples) such that:

(1) (All surviving strict orders credible) $\text{OP}(q^{\text{base}}) \subseteq C(q^{\text{base}})$.
(2) (No path-implied non-credible orders) In the directed graph whose vertices are the tuples that appear in q^{base} and whose edges are $C(q^{\text{base}})$, no directed path $e_i \rightsquigarrow e_j$ implies $(e_i \prec e_j)$ with $(e_i \prec e_j) \notin C(q^{\text{base}})$.

Problem: Given q, schema $\mathcal{R}$, and utilities (U^r, U^s), compute q^{base} as in Definition 4.5.

Algorithm: We present a graph-based algorithm (Algorithm 2) to compute q^{base} from q. For brevity, we represent a query as a partial order with sets of tuples (groups) in strict order $\mathrm{G}(q) = (g_1 \prec g_2 \prec \cdots \prec g_m)$, where each group g_i contains one or more tuples (e.g., $g_i = \{e_i \sim e_j\}$). We iteratively run Algorithm 1 on the current query q^{cur} (initially q) to obtain the credible orders C and set $\mathcal{NC} = \mathrm{OP}(q_{\text{cur}}) \setminus C$. If $\mathcal{NC} = \emptyset$ and no directed path in the graph *induced by C on the tuples of q^{cur}* implies a strict order outside C then the algorithm terminates and returns q^{cur} as q^{base}.

Otherwise, we pick a non-credible order $(e \prec e') \in \mathcal{NC}$ and merge the groups containing e and e'. If e and e' are in adjacent groups, we merge those two groups into a set. Else, we find a path $e \rightsquigarrow e'$ in the induced graph and merge all groups that contain nodes on this path into a single group. The algorithm repeats this iterative process until both conditions of Definition 4.5 are satisfied (Proposition 4.7).

Example 4.6. Consider the query $q(I) : e_1 \prec e_2 \prec e_3 \prec e_4$ from Example 4.4. The algorithm starts with $q_{\text{cur}} = q$ and $C = \{(e_1 \prec e_2), (e_2 \prec e_3), (e_3 \prec e_4)\}$. The non-credible set is $\mathcal{NC} = \{(e_1 \prec e_3), (e_1 \prec e_4), (e_2 \prec e_4)\}$. The algorithm picks $(e_1 \prec e_3)$ and merges the groups containing e_1 and e_3 into a single group, resulting in $q_{\text{cur}} : \{e_1, e_3\} \prec e_2 \prec e_4$. The algorithm runs Algorithm 1 again and finds $C = \{(e_1 \prec e_2), (e_2 \prec e_4)\}$ and $\mathcal{NC} = \{(e_1 \prec e_4)\}$. The algorithm picks $(e_1 \prec e_4)$ and merges the groups containing e_1 and e_4 into a single group, resulting in $q_{\text{cur}} : \{e_1, e_3, e_4\} \prec e_2$. The algorithm runs Algorithm 1 again and finds $C = \{(e_1 \prec e_2)\}$ and $\mathcal{NC} = \emptyset$. There are no directed paths in the graph induced by C that imply a strict order outside C. The algorithm halts and returns $q^{\text{base}} : \{e_1, e_3, e_4\} \prec e_2$.

Proposition 4.7. *Algorithm 2 halts after at most $k - 1$ merge operations and returns a list q^{base} that satisfies Definition 4.5. For k tuples, its running time is $O(k^3)$.*

Algorithm 2 Construct Base Credible Query

```
Input: submitted query q; schema R; utilities U^r, U^s
Output: q^base
q_cur ← q
loop
    _, C ← Algorithm 1(q_cur, R, U^r, U^s)
    NC ← OP(q_cur) \ C
    if NC = ∅ and no directed path in the graph induced by
    C on tuples of q^cur implies a strict order outside C then
        break                                   ▷ Def. 4.5 satisfied
    if there exists an adjacent boundary g_i ≺ g_{i+1} and some
    (e ≺ e') ∈ NC with e ∈ g_i, e' ∈ g_{i+1} then
        merge g_i and g_{i+1}; continue
    Build the directed graph G induced by C on tuples of q^cur
    Choose (e ≺ e') ∈ NC for which G contains a path e ⇝ e'
    Merge all groups that contain nodes on one such e ⇝ e'
    path into a single group in q^cur
return q^base ← q_cur
```

All possible equilibria: We now characterize the set of all possible equilibria that are possible because of the credible information in a user's query. We first define a class of reformulations of q_{base} that we call **super-rank queries** (Definition 4.8).

Definition 4.8. Let $q_1, q_2 \in \mathcal{Q}$ be queries of the same arity. For some instance I, let $q_1(I)$ and $q_2(I)$ denote the ranked lists produced by q_1 and q_2, respectively. We say that q_2 is a **super-rank query** of q_1 if:

(1) $\mathrm{elm}(q_1(I)) \subseteq \mathrm{elm}(q_2(I))$.
(2) For every pair of tuples $e_i, e_j \in \mathrm{elm}(q_1(I))$, if $e_i \sim e_j$ in $q_1(I)$, then $e_i \sim e_j$ in $q_2(I)$.
(3) For every tuple $e \in \mathrm{elm}(q_2(I)) \setminus \mathrm{elm}(q_1(I))$, there is no tuple $e_j \in \mathrm{elm}(q_1(I))$ such that $e \prec e_j$ in $q_2(I)$.

We denote the set of all super-rank queries of q_1 as $\mathcal{Q}_{\text{super}}(q_1)$.

Intuitively, executing the super-rank query ($q' \in \mathcal{Q}_{\text{super}}$) over some instance may (i) change a sub-list of ranked tuples to a **set** of tuples with the same position or (ii) **preserve** some of the orders (iii) **append** additional (ranked) tuples at the end of the list. Each query is a super-rank of itself.

Example 4.9. Suppose Algorithm 2 outputs the base credible query $q^{\text{base}} : e_1 \sim e_3 \prec e_4 \prec e_2$. Some super-rank queries of q^{base} are:

(a) set :	$e_1 \sim e_3 \sim e_4 \prec e_2$
(b) preserve order :	$e_1 \sim e_3 \prec e_4 \prec e_2$
(c) append :	$e_1 \sim e_3 \prec e_4 \prec e_2 \prec e_5$

Definition 4.10. A user strategy $P^r(\cdot|\cdot)$ is a **super-rank strategy** if for all intents τ and submitted queries q, we have $P^r(q \mid \tau) > 0$ iff q is a super-rank of q^{base}.

Lemma 4.11. *If q is credible (Definition 3.9), then every super-rank $q' \in \mathcal{Q}_{super}(q)$ is also credible.*

Theorem 4.12. *If the utility functions of the user and data source are supermodular and the user adopts a super-rank strategy, then the user and data source interaction has an equilibrium.*

Algorithm: The algorithm constructs all possible equilibria based on Definition 4.8. We provide the pseudocode in the Appendix. It explores the space in breadth-first order and deduplicates outputs. The result $\mathcal{E}$ is the set of all super-rank queries of q^{base}. We discuss the complexity of the algorithm in Proposition 4.13. The algorithm is inefficient and outputs a super-exponential number of equilibria. In Section 5, we present an algorithm that efficiently searches for a maximally-informative equilibrium.

Proposition 4.13. *Let m be the number of groups in q^{base} and let r be the number of tuples that may be appended. The algorithm outputs* super-exponential *equilibria and runs in super-exponential time in r.*

4.3 Achieving Desired Equilibria

The results in the preceding section indicate that to lead the reasoning to some equilibrium, it is sufficient for the user to choose certain types of strategies. *Nonetheless, users may prefer some equilibria over others.*

Example 4.14. Consider the $q^{\text{base}} : e_1 \sim e_3 \prec e_4 \prec e_2$ from Example 4.9. $q_1^{sup}(I)$: $e_1 \sim e_3 \sim e_4 \sim e_2$ and $q_2^{sup}(I)$: $e_1 \sim e_3 \sim e_4 \prec e_2$ The query that equally prefers all headphone brands (q_1^{sup}) and the one that has a higher preference for specific headphone brands (q_2^{sup}) are super-rank queries and both lead to equilibria in the interaction.

As illustrated in Example 4.14, using some super-rank queries, the user can overstate their preference for some tuples that might otherwise not be returned by the data source. However, a super-rank strategy that maps an intent to a query that places too many tuples in higher positions than their true positions in the intent or adds too many tuples to the intent may reduce the expected posterior of all tuples. For example, in Example 3.3, again, assume that the data source has only four headphones. If the user submits a query that returns all headphones in the data source, the expected posterior of every tuple will be $\frac{1}{2}$. Since the data source returns only tuples where preference values are above 0.5, it will return only one tuple. Thus, the user strategy must find the right amount of modification to the intent to offset the data source bias and maximize extracted information.

In a fully informative equilibrium, the data source returns all tuples relevant to the intent behind the submitted user's query, which is desirable for users:

Definition 4.15. An equilibrium $(P^r(.|.), P^s(.|.))$ is **fully informative** if for every query q and intent τ such that $P^r(q|\tau) > 0$, τ is a super-rank of β for all β where $P^s(\beta|q) > 0$.

The data source might return some tuples that are not relevant to the user's intent. However, the user can filter out these additional tuples by post-processing the returned results from the data source.

To achieve a fully informative equilibrium, the user must convince the system that all its desired tuples are highly relevant to their intent. But, as we have explained in the preceding paragraphs, it might not always be possible to achieve that. Thus, one might aim for an equilibrium that *extracts as much useful information as possible from the data source.*

Definition 4.16. An equilibrium $E_{\max} = (P^r_{\max}(\cdot|\cdot), P^s_{\max}(\cdot|\cdot))$ is **maximally informative** over a set of equilibria $\mathcal{E}$ if, for every equilibrium $E = (P^r(\cdot|\cdot), P^s(\cdot|\cdot)) \in \mathcal{E}$ and every query q with $P^s_{\max}(\beta_{\max}|q) > 0$, the super-rank $\beta_{\max}$ is at least as informative as every β for which $P^s(\beta|q) > 0$.

5 Maximally Informative Strategy

In Section 3.4, we prove that users can leverage super-rank strategies to attain informative equilibria despite conflicts of interest. In this section, we propose algorithms for finding queries that led to a maximally informative equilibrium.

Problem: Given the user's intent query (Section 2.3), users' and system's utility functions U^r and U^s and schema $\mathcal{R}$ as **input**, our goal is to **output** a query $q^\star$ that leads to a maximally informative equilibrium (Definition 4.16).

$$q^\star = \arg\max_{q'} \mathbb{E}[U^r(\tau, \beta) \mid q'],$$

Brute-Force Algorithm: Algorithm 3 presents the exhaustive search workflow. The algorithm enumerates all possible super-rank queries ($\mathcal{Q}_{\text{super}}$). For each query, it gets the system's interpretation (β), and then computes the user's utility $U^r(q^{sup}, \beta)$, finally returning one that maximizes user utility.

Algorithm 3 Finding Maximally Informative Query

Input: τ, schema ($\mathcal{R}$), utilities U^r, U^s
Output: Maximally informative query $q^\star$.
$q^{\text{base}} \leftarrow$ Algorithm 2 (τ, schema ($\mathcal{R}$), utilities U^r, U^s)
$\beta_{q^{\text{base}}} \leftarrow \arg\max_{\beta'} \mathbb{E}[U^s(\tau, \beta') \mid q^{\text{base}}]$
$U^r_{org} \leftarrow$ Compute original user's utility $U^r(q^{\text{base}}, \beta)$
$q^\star \leftarrow q^{\text{base}}$ ▹ Initialize with base credible query
for each $q^{sup} \in \mathcal{Q}_{\text{super}}(q^{\text{base}})$ **do** ▹ All super-rank queries
  $\beta \leftarrow \arg\max_{\beta'} \mathbb{E}[U^s(\tau, \beta') \mid q^{sup}]$
  $U^r_{new} \leftarrow$Compute user's utility $U^r(q^{sup}, \beta)$
  if $U^r_{new} > U^r_{org}$ **then** ▹ More informative query found
    $U^r_{org} \leftarrow U^r_{new}$
    $q^\star \leftarrow q^{sup}$
return $q^\star$

Complexity: Recall from Definition 4.8, the set of all possible super-rank queries ($\mathcal{Q}_{\text{super}}$) is created by selectively merging adjacent positions from the base credible query q^{base}. Each pair of adjacent positions can either merge or remain separate, generating unique super-rank reformulations.

In previous sections (Sections 3 and 4.2), the input to the algorithms is the original query q, which is a total-order query containing n distinct positions corresponding exactly to distinct domain values of the ranking attribute. However, the current algorithm (Algorithm 3) uses the base credible query q^{base} (Definition 4.5) as input, which typically results from merging positions and therefore has fewer distinct ranking positions. Denote this reduced number of distinct positions as $m \leq n$.

Since each of the $m-1$ adjacent position pairs independently offers two choices (either merge or remain separate), the total number of possible super-rank queries is$|\mathcal{Q}_{\text{super}}| = 2^{m-1}$. Thus, Algorithm 3 exhibits exponential time complexity $O(2^{m-1})$.

Theorem 5.1. *The maximally informative strategy problem, even with systems' utility U^s as a constant-time but arbitrary oracle, is NP-complete*

We prove (i) membership in NP and (ii) NP-hardness by a Karp reduction from the NP-complete Subset-Sum problem. Subset-Sum says given positive integers $w_1, \ldots, w_n$ and a target W, decide whether there exists $S \subseteq \{1, \ldots, n\}$ with $\sum_{j \in S} w_j = W$. We reduce our problem to Subset-Sum by realizing that finding a maximally informative superrank requires identifying subsets of q^{base} to merge, and each w is the user's utility U^r.

5.1 Efficient Maximally Informative Strategy

The exhaustive algorithm proposed in the previous section has a high computation cost. It quickly becomes intractable. In this section, we propose an efficient algorithm that finds a maximally informative query in polynomial time. The key insight is that the

user's expected utility can be decomposed into additive contributions from contiguous segments of the base credible query (Definition 4.4). This allows us to use dynamic programming to efficiently search for the optimal way to merge groups in the base credible query and therefore find the super-rank query (Definition 4.9) that maximizes the user's expected utility.

Merge query: Let q^{base} (Algorithm 2), as in Section 4, we can denote the base credible query as:

$$q^{\text{base}}: \quad g_1 \prec g_2 \prec \cdots \prec g_m.$$

A super-rank of q^{base} is obtained by merging adjacent groups into one tied group.

Definition 5.2. Given q with groups $g_1 \prec \cdots \prec g_m$ and indices $1 \le i \le j \le m$, the **merge query** $q_{i\sim j}$ replaces the consecutive groups $g_i, \ldots, g_j$ with their union as a single tied group; all other orders are unchanged. Let $Q_{\text{merge}}(q)$ be the set of all such reformulations. Clearly, $Q_{\text{merge}}(q) \subseteq Q_{\text{super}}(q)$.

Lemma 5.3. *Let $q^{\text{base}}: g_1 \prec \cdots \prec g_m$ and consider any partition of the groups into consecutive runs $\Pi = \{[i_1, j_1], \ldots, [i_t, j_t]\}$. For each run $[i, j]$, let $n_{i:j} = \left|g_i \cup \cdots \cup g_j\right|$ (number of tuples in the run) and let $\overline{u}(i, j)$ be the* expected per-tuple user utility *when exactly $g_i, \ldots, g_j$ are merged (and the system best-responds to that merge). Then there is a run score $\Phi(i, j) = n_{i:j} \cdot \overline{u}(i, j)$ such that the user's expected utility for the whole query equals*

$$\sum_{\ell=1}^{t} \Phi(i_\ell, j_\ell) \; + \; C,$$

where C is a constant that does not depend on how we cut the runs.

Intuitively, the lemma says that the user's total expected utility decomposes into independent contributions from each run of merged groups. Take any chunk (run) of consecutive groups and merge them: the contribution of that chunk is just the number of tuples in the chunk times the expected per-tuple utility when that chunk is merged.

Algorithm: We can now efficiently find a maximally informative super-rank by dynamic programming (DP) over prefixes of $g_1, \ldots, g_m$. The DP chooses where to cut the groups into runs, where each run is equivalent to a merge query (Definition 5.2). We compute the run scores $\Phi(i, j)$ for all $1 \le i \le j \le m$ based on Lemma 5.3, and then use the DP to find the optimal partition of $g_1, \ldots, g_m$ into runs that maximizes the total score. Because the objective is a sum of run scores (Lemma 5.3), choosing the optimal super-rank reduces to cutting the groups into consecutive runs with maximum total score.

Theorem 5.4. *Let $\text{OPT}(j)$ denote the maximum total utility for the prefix $g_1, \ldots, g_j$. Then*

$$\text{OPT}(0) = 0, \qquad \text{OPT}(j) = \max_{1 \le i \le j} \left\{\text{OPT}(i-1) + \Phi(i, j)\right\}.$$

Any optimal solution on the prefix $[1..j]$ ends with some last run $[i..j]$. By Lemma 5.3, its contribution is $\Phi(i, j)$, and the best possible contribution before it is $\text{OPT}(i-1)$. Maximizing over the choice of i yields the recurrence. Conversely, picking the maximum i for each j constructs an optimal partition.

Algorithm 4 MaxInfoMergeDP (no append)

Input: intent τ, schema $\mathcal{R}$, utilities U^r, U^s
Output: Maximally informative query $q^\star$
$q^{\text{base}} \leftarrow$ Algorithm 2$(\tau, \mathcal{R}, U^r, U^s)$
Let $q^{\text{base}}: g_1 \prec \cdots \prec g_m$
▹ Compute run scores
$n[k] \leftarrow |g_k|$ for $k = 1..m$; $\quad P[j] \leftarrow \sum_{k=1}^{j} n[k]$ ▹ prefix counts
for $1 \le i \le j \le m$ **do**
  $n_{i:j} \leftarrow P[j] - P[i-1]$
  $q_{i\sim j} \leftarrow$ merge $g_i, \ldots, g_j$ in q^{base} ▹ Definition 5.2
  $\beta(q_{i\sim j}) \leftarrow \arg\max_{\beta'} \mathbb{E}[U^s(\tau, \beta') \mid q_{i\sim j}]$
  $\overline{u}^r(i, j) \leftarrow \mathbb{E}\left[u_e^r(\Theta_\tau(e), \Theta_{\beta(q_{i\sim j})}(e)) \,\middle|\, e \text{ placed in run } [i, j]\right]$
  $\Phi(i, j) \leftarrow n_{i:j} \cdot \overline{u}^r(i, j)$ ▹ Lemma 5.3
▹ DP over prefixes
$\text{DP}[0] \leftarrow 0$; $\text{parent}[0] \leftarrow 0$
for $j = 1$ to m **do**
  $\text{DP}[j] \leftarrow -\infty$; $\text{parent}[j] \leftarrow \perp$
  for $i = 1$ to j **do**
    if $\text{DP}[i-1] + \Phi(i, j) > \text{DP}[j]$ **then**
      $\text{DP}[j] \leftarrow \text{DP}[i-1] + \Phi(i, j)$; $\text{parent}[j] \leftarrow i$
▹ Backtracking to reconstruct optimal runs
$j \leftarrow m$; initialize empty list
while $j > 0$ **do**
  $i \leftarrow \text{parent}[j]$; prepend merged group $g_i \cup \cdots \cup g_j$; $j \leftarrow i-1$
Form $q^\star$ by merging exactly the reconstructed runs
return $q^\star$

Proposition 5.5. *Under additivity and i.i.d., Algorithm 4 outputs a maximally informative super-rank of q^{base} and preserves credibility. It runs in $O(m^2)$ time for the DP plus the cost of tabulating $\Phi(i, j)$; with cached summaries, this is $O(m^2)$ overall.*

5.2 Guarantees to avoid Algorithms

6 Experiments

7 Related Work

Privacy Preserving Querying Users might use tools and services, e.g., Tor (*torproject.org*), that allow them to hide their identity from data systems. Nevertheless, users might want both their identity and intent to be hidden from the data system. For instance, consider a retail company that searches a database of available lands for sale in a region to build a new branch or a researcher that searches a database of patents for inventions similar to her unregistered ones. In these cases, a data system can use the information in users' queries and eventually use it against their interests, e.g., by increasing the price on available lands or discovering and patenting the researcher's idea. Also, many popular data services, e.g., Google search engine, do not allow users to submit their queries via such services. These methods might also add significant communication delays. Furthermore, these services might still leak users' querying data to third-parties [13].

Hence, there is often no protection for users' data when they query data systems. Since users usually have no choice but to query

these systems to find relevant information, it is vital to preserve user's privacy in this settings at least to some extent. This is challenging as not only does not the data system implement any privacy preserving mechanism but also it is in their interest to reveal and take advantage of users' data. Thus, it is not clear whether the users can achieve complete privacy while having informative interaction with the system.

If users would like to receive accurate answers, they have to share their intents with the system completely. However, the users might preserve some level of privacy if they are willing to reduce the accuracy of their answers to some degree and share information about their intent with the system strategically. A user can modify their query to represent an information need different from their original intent to hide their true information need from the system. Of course, this approach will not result in an informative communication as the returned results will not satisfy the user's true information need. The user, however, might be able to change their query such that it is sufficiently similar to the true intent to return some informative results and sufficiently different with that intent to provide some degree of privacy.

Bias in returned results

Query Reformulation Prior work on reformulating top-k ranking queries as range selection queries for query optimization [5, 12]

Evidence of strategic behavior Survey on lab studies demonstrating similar human's strategic behavior in games [4] Strategic behavior with ranking messages in real-state listing services [20] Study millions of interactions in Ebay listing to show that user's may stategically manipulate their listing prices and achieve better outcomes [?]

References

[1] Amazon. 2025. Amazon selling fees. https://sell.amazon.com/pricing Accessed: 2025-01-21.

[2] Simon P. Anderson, Jacob K. Goeree, and Roald Ramer. 1997. Location, Location, Location. *Journal of Economic Theory* 77, 1 (November 1997), 102–127. https://ideas.repec.org/a/eee/jetheo/v77y1997i1p102-127.html

[3] Miguel E. Andrés, Nicolás E. Bordenabe, Konstantinos Chatzikokolakis, and Catuscia Palamidessi. 2013. Geo-indistinguishability: differential privacy for location-based systems. In *Proceedings of the 2013 ACM SIGSAC Conference on Computer & Communications Security* (Berlin, Germany) *(CCS '13)*. Association for Computing Machinery, New York, NY, USA, 901–914. doi:10.1145/2508859.2516735

[4] Andreas Blume, Ernest K. Lai, and Wooyoung Lim. 2020. Chapter 13 Strategic information transmission: a survey of experiments and theoretical foundations. In *Handbook of Experimental Game Theory*. Edward Elgar Publishing, Cheltenham, UK. doi:10.4337/9781785363337.00022

[5] Nicolas Bruno, Surajit Chaudhuri, and Luis Gravano. 2002. Top-k selection queries over relational databases: Mapping strategies and performance evaluation. *ACM Trans. Database Syst.* 27, 2 (June 2002), 153–187. doi:10.1145/568518.568519

[6] L. Elisa Celis, Anay Mehrotra, and Nisheeth K. Vishnoi. 2020. Interventions for ranking in the presence of implicit bias. In *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency* (Barcelona, Spain) *(FAT* '20)*. Association for Computing Machinery, New York, NY, USA, 369–380. doi:10.1145/3351095.3372858

[7] L. Elisa Celis, Damian Straszak, and Nisheeth K. Vishnoi. 2018. Ranking with Fairness Constraints. In *45th International Colloquium on Automata, Languages, and Programming (ICALP 2018) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 107)*, Ioannis Chatzigiannakis, Christos Kaklamanis, Dániel Marx, and Donald Sannella (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 28:1–28:15. doi:10.4230/LIPIcs.ICALP.2018.28

[8] Archishman Chakraborty and Rick Harbaugh. 2007. Comparative cheap talk. *Journal of Economic Theory* 132, 1 (2007), 70–94. doi:10.1016/j.jet.2005.07.003

[9] Reynold Cheng, Yu Zhang, Elisa Bertino, and Sunil Prabhakar. 2006. Preserving user location privacy in mobile data management infrastructures. In *Proceedings of the 6th International Conference on Privacy Enhancing Technologies* (Cambridge, UK) *(PET'06)*. Springer-Verlag, Berlin, Heidelberg, 393–412. doi:10.1007/11957454_23

[10] Benny Chor, Eyal Kushilevitz, Oded Goldreich, and Madhu Sudan. 1998. Private Information Retrieval. *J. ACM* 45, 6 (nov 1998), 965–981. https://doi.org/10.1145/293347.293350

[11] Vincent P. Crawford and Joel Sobel. 1982. Strategic Information Transmission. *Econometrica* 50, 6 (1982), 1431–1451. http://www.jstor.org/stable/1913390

[12] Donko Donjerkovic and Raghu Ramakrishnan. 1999. Probabilistic Optimization of Top N Queries. In *Proceedings of the 25th International Conference on Very Large Data Bases (VLDB '99)*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 411–422.

[13] DuckDuckGo. 2023. DuckDuckGo browser allows Microsoft trackers due to search agreement. https://www.bleepingcomputer.com/news/security/duckduckgo-browser-allows-microsoft-tracke Accessed: 2023-04-15.

[14] Sidney Fussell. 2019. Is Amazon's search engine biased? It's hard to tell. *Atlantic* (18 September 2019).

[15] Baik Hoh and Marco Gruteser. 2005. Protecting Location Privacy Through Path Confusion. *Securecomm*, 194– 205. doi:10.1109/SECURECOMM.2005.33

[16] HPCwire. 1997. IBM launches free WWW-based U.S. patent search. https://www.hpcwire.com/1997/01/17/ibm-launches-free-www-based-u-s-patent-search/

[17] Hongbo Jiang, Jie Li, Ping Zhao, Fanzi Zeng, Zhu Xiao, and Arun Iyengar. 2021. Location Privacy-preserving Mechanisms in Location-based Services: A Comprehensive Survey. *ACM Comput. Surv.* 54, 1, Article 4 (jan 2021), 36 pages. doi:10.1145/3423165

[18] Werner Kießling and Gerhard Köstler. 2002. Chapter 91 - Preference SQL — Design, Implementation, Experiences. In *VLDB '02: Proceedings of the 28th International Conference on Very Large Databases*, Philip A. Bernstein, Yannis E. Ioannidis, Raghu Ramakrishnan, and Dimitris Papadias (Eds.). Morgan Kaufmann, San Francisco, 990–1001. doi:10.1016/B978-155860869-6/50098-6

[19] Nancy Lambert. 1997. But what's in it for IBM? 'Free' patents on the net. *Searcher* 5 (September 1997). Issue 8. link.gale.com/apps/doc/A19782789/EAIM?u=anon~1bf0a033&sid=sitemap&xid=12b4fa7d

[20] Lingxiao Li and Jia Xie. 2025. Comparative Cheap Talk: Evidence from the Housing Market. *International Real Estate Review* 28, 2 (None 2025), 117–148. doi:None

[21] Xiao Lin and Ce Liu. 2022. Credible Persuasion. In *Proceedings of the 23rd ACM Conference on Economics and Computation* (Boulder, CO, USA) *(EC '22)*. Association for Computing Machinery, New York, NY, USA, 469. doi:10.1145/3490486.3538264

[22] Dana Mattioli. 2019. Amazon Changed Search Algorithm in Ways That Boost Its Own Products. *Wall Street Journal* (16 September 2019). https://www.wsj.com/articles/amazon-changed-search-algorithm-in-ways-that-boost-its-own-p

[23] Niccolò Meneghetti, Denis Mindolin, Paolo Ciaccia, and Jan Chomicki. 2015. Output-sensitive Evaluation of Prioritized Skyline Queries. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data* (Melbourne, Victoria, Australia) *(SIGMOD '15)*. Association for Computing Machinery, New York, NY, USA, 1955–1967. doi:10.1145/2723372.2723736

[24] Tripp Mickle. 2019. Apple Dominates App Store Search Results, Thwarting Competitors. *Wall Street Journal* (23 July 2019). https://www.wsj.com/articles/apple-dominates-app-store-search-results-thwarting-competitors-

[25] Mohamed F. Mokbel, Chi-Yin Chow, and Walid G. Aref. 2006. The new Casper: query processing for location services without compromising privacy. In *Very Large Data Bases Conference*. https://api.semanticscholar.org/CorpusID:6583100

[26] Einar Mykletun, Maithili Narasimha, and Gene Tsudik. 2006. Authentication and Integrity in Outsourced Databases. *ACM Trans. Storage* 2, 2 (may 2006), 107–138. doi:10.1145/1149976.1149977

[27] MySQL. [n. d.]. MySQL. https://dev.mysql.com/doc/refman/8.4/en/case.html. Accessed: 2025-02-10.

[28] Jack Nicas and Keith Collins. 2019. How Apple's apps topped rivals in the App Store it controls. *The New York Times* (9 September 2019). https://www.nytimes.com/interactive/2019/09/09/technology/apple-app-store-competition.html

[29] Oracle. [n. d.]. Oracle. https://docs.oracle.com/cd/B13789_01/appdev.101/b10807/13_elems004.htm. Accessed: 2025-02-10.

[30] D. Kossmann S. Borzsony and K. Stocker. 2001. The Skyline Operator. In *Proceedings of the 17th International Conference on Data Engineering (ICDE '01)*. IEEE Computer Society, USA, 421.

[31] SQLServer. [n. d.]. SQLServer. https://learn.microsoft.com/en-us/sql/t-sql/language-elements/case Accessed: 2025-02-10.

[32] Stephen Tu, M. Frans Kaashoek, Samuel Madden, and Nickolai Zeldovich. 2013. Processing Analytical Queries over Encrypted Data. *Proc. VLDB Endow.* 6, 5 (mar 2013), 289–300. doi:10.14778/2535573.2488336

[33] Meike Zehlike, Ke Yang, and Julia Stoyanovich. 2022. Fairness in Ranking, Part I: Score-Based Ranking. *ACM Comput. Surv.* 55, 6, Article 118 (dec 2022), 36 pages.

doi:10.1145/3533379

[34] Meike Zehlike, Ke Yang, and Julia Stoyanovich. 2022. Fairness in Ranking, Part II: Learning-to-Rank and Recommender Systems. *ACM Comput. Surv.* 55, 6, Article 117 (dec 2022), 41 pages. doi:10.1145/3533380

[35] Karen Zraik. 2024. Farmers Sue Over Deletion of Climate Data From Government Websites. *The New York Times* (25 February 2024). https://www.nytimes.com/2025/02/24/climate/agriculture-farmer-website-data-lawsuit.html

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009